

Decoder Card — reading the banner's binary

Each column of eight squares is one character. ☐ white = 1, ☒ black = 0. You don't have to memorize anything — every square is just a number, and you add them up.

Why it's built this way. ASCII was standardized in **1963** for teleprinters, back when every bit cost real money to send down a wire — seven bits covered all 128 characters anyone needed, so nobody paid for an eighth. When the 1967 revision added lowercase letters, it parked each one exactly **32** above its capital, which is why a single square decides the case: pressing **Shift** is really just subtracting 32. The 8-bit byte only became standard *afterward*, leaving ASCII's seven bits sitting inside it with one square left over — which early equipment often spent on an error check.

WHAT EACH ROW IS WORTH

Going *up* the column, each square is worth double the one below it. Add the values of the **white** squares.

Row	Worth	What it tells you
row 1	128	Always black. ASCII really only needs 7 squares, so this spare one is never used.
row 2	64	White on every letter — it's what says " a letter lives here. "
row 3	32	The case square: black = CAPITAL, white = lowercase.
row 4	16	These five carry the rest of the number.
row 5	8	
row 6	4	
row 7	2	
row 8	1	

THE METHOD

Read the column of squares as **eight 0s and 1s**

↓

Add up what the white squares are worth

↓

Look that **number** up on the **ASCII table**

That's the whole procedure. The number you land on is the character — no letter-counting, no decoder ring.

Two sanity checks while you work. Every letter lands between **65** and **122**, and every number in that range contains the 64 — so **row 2 is white on every letter you'll meet**. And if row 2 goes black, you are almost certainly looking at a **space**, which is 32 exactly.

TWO WORKED EXAMPLES

A capital letter

128

64

32

16

8

4

2

1

+ 64

+ 4

+ 2

+ 1

= 71

Row 1 black, as always. Row 2 white, so this is a letter. Row 3 is **black** — that makes it a **CAPITAL**. Now just add the white squares going down: 64, then 4, 2 and 1.

binary **01000111**
number **71** → ASCII table says **G**

A lowercase letter

128

64

32

16

8

4

2

1

+ 64

+ 32

+ 16

+ 4

+ 2

+ 1

= 119

Same start: row 1 black, row 2 white. But row 3 is **white** here, which adds **32** and makes it **lowercase**. Add everything white: 64, 32, 16, 4, 2 and 1.

binary **01110111**
number **119** → ASCII table says **w**